

Name: Kunchala Naga Chaitanya Sphoorthy

Roll Number: 2420030510

Experiment-6

DOCKER CONTAINERIZATION

1. Before You Start

Install Docker Desktop---<https://docs.docker.com/desktop/setup/install/windows-install/>

2. Five Simple Words

Image: A packaged snapshot of an application and everything it needs to run.

Container: A running instance of an image.

Dockerfile: A text file with the instructions used to build an image.

Build: The process of turning a Dockerfile into an image.

Port mapping: Connecting a port on the host machine to a port inside the container.

3. Workflow Used in This Practical

The practical uses a simple Flask web application, containerized and run with Docker.

docker-sample-app

├─ app.py

├─ Dockerfile

└─ requirements.txt

The screenshots below show the complete workflow from creating the project files, building the Docker image, running the container, verifying the application in the browser, checking logs, and stopping the container.

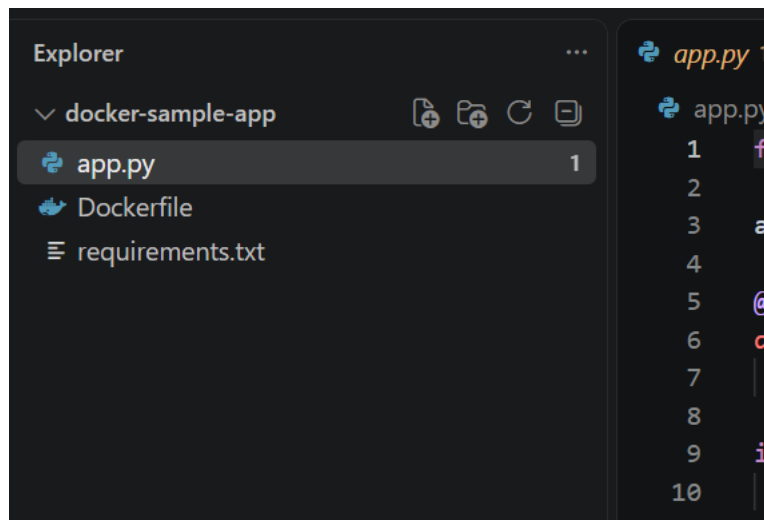
Part A — Screenshot-Based Practical (Steps 1–11)

Each step below shows exactly what appears on screen, a simple explanation of what is happening, the Docker command used (where applicable), and the corresponding screenshot from the practical session.

Step 1. Create the Project Structure

What you see: The terminal shows the project folder `docker-sample-app` containing three files: `app.py`, `Dockerfile`, and `requirements.txt`.

Simple explanation: Before building a Docker image, the project files need to exist in one folder. This folder will later be copied into the image.

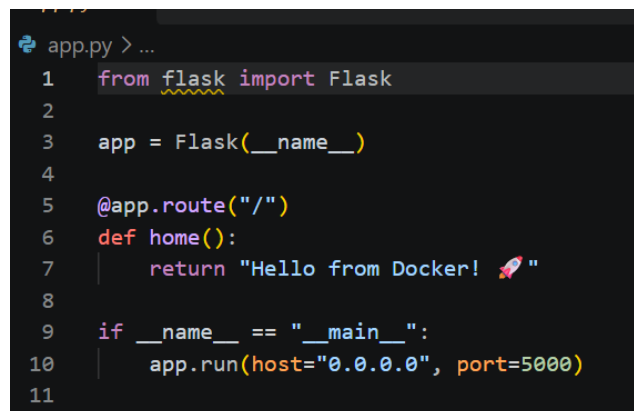


Screenshot 1 — Step 1

Step 2. Write the Application Code (app.py)

What you see: The editor shows `app.py` containing a small Flask application with one route ('/') that returns a welcome message.

Simple explanation: This is the actual program that will run inside the container. Flask starts a small web server on port 5000 and responds with "Hello from Docker!" when the home page is visited.

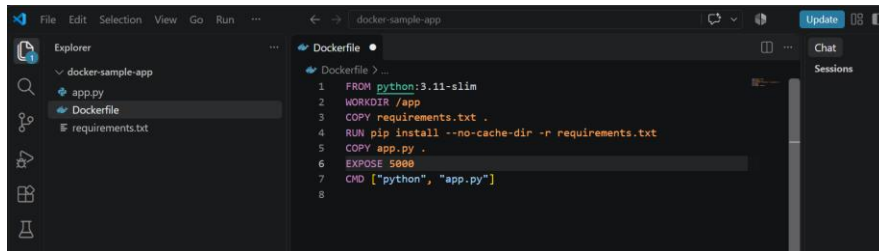


Screenshot 2 — Step 2

Step 3. Create the Dockerfile

What you see: The editor shows a file named Dockerfile with instructions: a base Python image, a working directory, dependency installation, copying the app, exposing a port, and the startup command.

Simple explanation: A Dockerfile is a recipe Docker follows to build an image. Each line is one instruction: choose a base image, set up the folder, install requirements, copy the code in, open a network port, and say how to start the app.

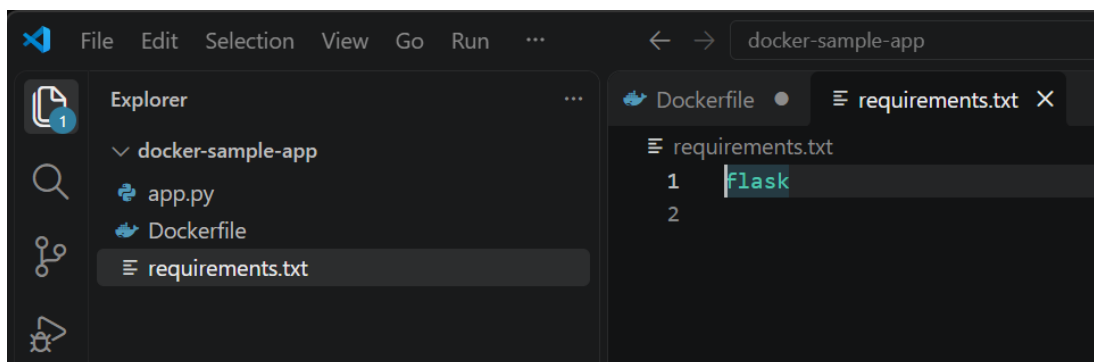


Screenshot 3 — Step 3

Step 4. Create requirements.txt

What you see: The editor shows requirements.txt containing a single line: flask.

Simple explanation: This file lists the Python packages the app depends on. Docker reads it during the build and installs everything listed, so Flask is available inside the container.



Screenshot 4 — Step 4

Step 5. Build the Docker Image

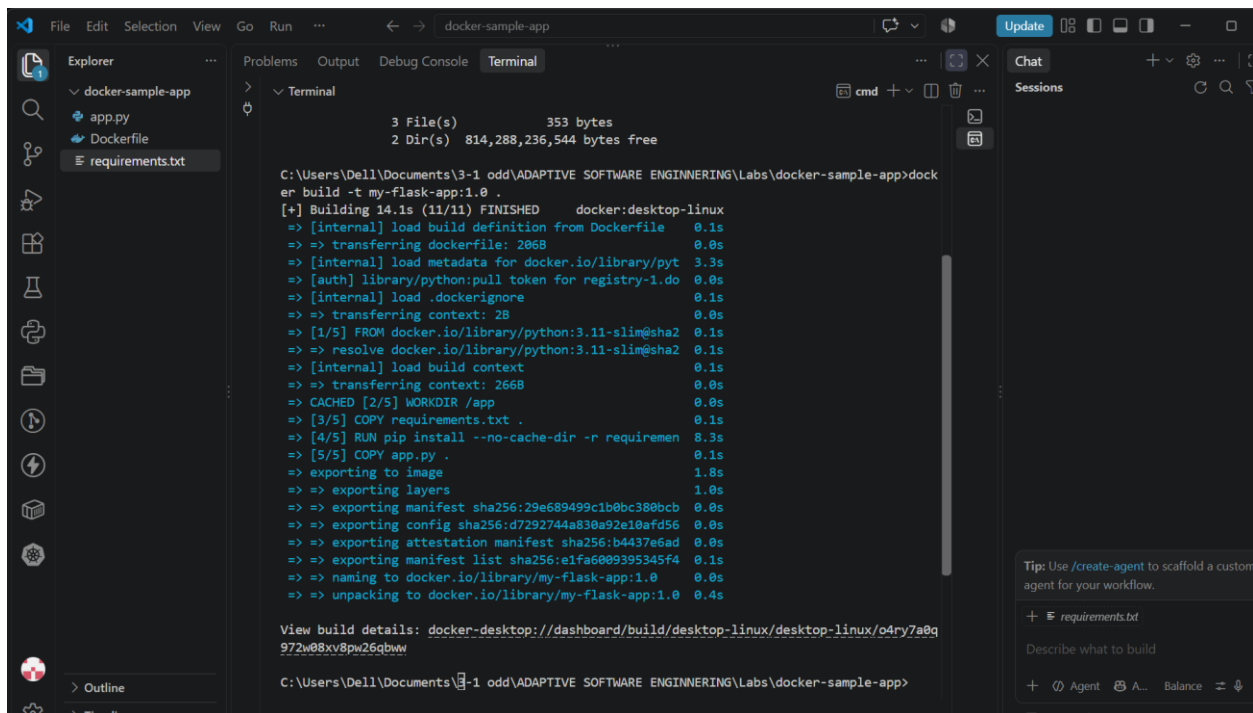
What you see: The terminal shows docker build -t my-flask-app:1.0 . running through each build stage and finishing successfully, ending with the image being named my-flask-app:1.0.

Simple explanation: This command reads the Dockerfile and builds an image step by step: loading the base Python image, installing requirements, copying in the app, and packaging everything together as a reusable image.

Command:

```
docker build -t my-flask-app:1.0 .
```

Purpose: Builds a Docker image from the Dockerfile and tags it my-flask-app:1.0.



Screenshot 5 — Step 5

Step 6. Verify the Docker Image

What you see: The terminal shows docker images listing my-flask-app:1.0 alongside the base python:3.11-slim image, with their image IDs, creation time, and size.

Simple explanation: This confirms the build worked. The new image now exists locally and can be used to start containers.

Command:

```
docker images
```

Purpose: Lists all Docker images currently stored on the machine.

```
C:\Users\Dell\Documents\3-1 odd\ADAPTIVE SOFTWARE ENGINEERING\Labs\docker-sample-app>docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	In Use
docker/desktop-cloud-provider-kind:v0.6.0	46d51b990e97	613MB	168MB	U
docker/desktop-containerd-registry-mirror:v0.0.4	1de142695bb0	52.8MB	16.9MB	U
envoyproxy/envoy:v1.36.7	29d778ba078e	253MB	63.9MB	
gcr.io/k8s-minikube/kicbase:v0.0.50	b97074569ae9	1.94GB	545MB	
gcr.io/k8s-minikube/kicbase@sha256:eb4fec00e8ad70adf8e6436f195cc429825ffb85f95afcdb5d8d9deb576f3e93	eb4fec00e8ad	1.94GB	545MB	U
hello-world:latest	7f4da0fc94bc	25.9kB	9.49kB	U
kindest/node:v1.36.1	3489c7674813	1.31GB	383MB	U
my-flask-app:1.0	e1fa60093953	211MB	51.3MB	
retailiq-retailiq:latest	a4f3843389b7	1.33GB	305MB	U
student-grade-app:latest	f6472b235c99	464MB	128MB	

```
C:\Users\Dell\Documents\3-1 odd\ADAPTIVE SOFTWARE ENGINEERING\Labs\docker-sample-app>
```

Screenshot 6 — Step 6

Step 7. Run the Container

What you see: The terminal shows `docker run -d -p 5000:5000 --name flask-container my-flask-app:1.0`, followed by a long container ID.

Simple explanation: This starts a container from the image in the background (-d), maps port 5000 on the machine to port 5000 inside the container (-p), and gives the container a friendly name.

Command:

```
docker run -d -p 5000:5000 --name flask-container my-flask-app:1.0
```

Purpose: Starts a new container from the my-flask-app:1.0 image and exposes it on port 5000.

```
C:\Users\Dell\Documents\3-1 odd\ADAPTIVE SOFTWARE ENGINEERING\Labs\docker-sample-app>docker run -d -p 5000:5000 --name flask-container my-flask-app:1.0
78f4302e79ff1d7c4e2e380a75e929aa2874c631ee4796a0cefd10eed7eb74e

C:\Users\Dell\Documents\3-1 odd\ADAPTIVE SOFTWARE ENGINEERING\Labs\docker-sample-app>
```

Screenshot 7 — Step 7

Step 8. Check the Running Container

What you see: The terminal shows `docker ps` listing the running container: its ID, image, command, uptime, status, port mapping, and name (flask-container).

Simple explanation: This confirms the container is actually running and shows which port it is reachable on.

Command:

```
docker ps
```

Purpose: Lists all currently running containers.

```
C:\Users\Dell\Documents\3-1 odd\ADAPTIVE SOFTWARE ENGINEERING\Labs\docker-sample-app>docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
78f4302e79ff	my-flask-app:1.0	"python app.py"	3 minutes ago	Up 3 minutes	0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp

```
flask-container
```

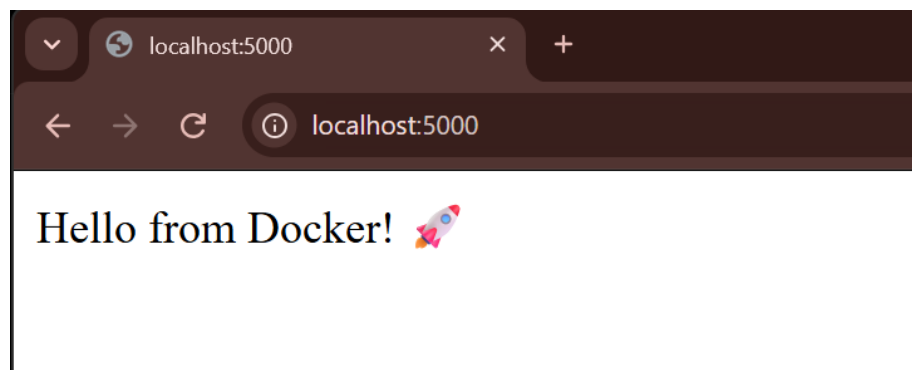
```
C:\Users\Dell\Documents\3-1 odd\ADAPTIVE SOFTWARE ENGINEERING\Labs\docker-sample-app>
```

Screenshot 8 — Step 8

Step 9. Access the Application in the Browser

What you see: The browser is open at `http://localhost:5000` and displays the message "Hello from Docker!" with a rocket emoji.

Simple explanation: This proves the whole chain worked: the container is running, Flask is serving requests, and the app is reachable from outside the container through the mapped port.



Screenshot 9 — Step 9

Step 10. View Container Logs

What you see: The terminal shows docker logs flask-container, printing Flask's startup messages, a development-server warning, the addresses it is listening on, and an incoming GET request.

Simple explanation: Logs show what is happening inside the container in real time. They are useful for confirming the app started correctly and for troubleshooting requests.

Command:

```
docker logs flask-container
```

Purpose: Shows the console output produced by the application inside the container.

```

C:\Users\Dell\Documents\3-1 odd\ADAPTIVE SOFTWARE ENGINEERING\Labs\docker-sample-app>dock
er logs flask-container
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a pr
oduction WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.2:5000
Press CTRL+C to quit
172.17.0.1 - - [26/Sep/2026 03:44:27] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [26/Sep/2026 03:44:27] "GET /favicon.ico HTTP/1.1" 404 -

C:\Users\Dell\Documents\3-1 odd\ADAPTIVE SOFTWARE ENGINEERING\Labs\docker-sample-app>

```

Screenshot 10 — Step 10

Step 11. Stop the Container

What you see: The terminal shows `docker stop flask-container`, which prints the container name, followed by `docker ps` showing an empty list.

Simple explanation: This stops the running container. `docker ps` afterward confirms that no containers are running anymore.

Command:

```
docker stop flask-container
```

Purpose: Stops the running container named `flask-container`.

```

C:\Users\Dell\Documents\3-1 odd\ADAPTIVE SOFTWARE ENGINEERING\Labs\docker-sample-app>dock
er stop flask-container
flask-container

C:\Users\Dell\Documents\3-1 odd\ADAPTIVE SOFTWARE ENGINEERING\Labs\docker-sample-app>dock
er ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS        NAMES
C:\Users\Dell\Documents\3-1 odd\ADAPTIVE SOFTWARE ENGINEERING\Labs\docker-sample-app>

```

Screenshot 11 — Step 11

Final Revision

Remember the main flow: create the project files → write the Dockerfile and requirements → build the image → verify the image exists → run a container from it → confirm it is running → open the app in the browser → check the logs → stop the container.

Most Important Commands

docker build -t <name>:<tag> . – Builds an image from the Dockerfile in the current folder.

docker images – Lists images stored locally.

docker run -d -p <host>:<container> --name <name> <image> – Starts a container from an image.

docker ps – Lists running containers.

docker logs <container> – Shows console output from a container.

docker stop <container> – Stops a running container.